In [1]:
```
# Behzad Ghabaei
# CS 82 A - Data Science
# module7_lab3.ipynb
# Text Book Problem 8.1
# Titanic: who survived?
# Instructor Seno
# 10/13/2026
```

Step 1: Load and Inspect the Data

In [2]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier # or KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load the training dataset
df = pd.read_csv("titanic_train.csv")
```

Step 2: Feature Preparation & Cleaning

Machine learning models require numeric inputs and cannot process missing values directly. You need to convert the text column Sex into numerical values and handle the blank spaces in Age.

1. Encode Categorical Data: Convert male to 0 and female to 1.

2. Impute Missing Values: Calculate the median age from the data and use it to fill any missing age entries.

In [3]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    str    
 4   Sex          891 non-null    str    
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    str    
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str    
 11  Embarked     889 non-null    str    
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
```

In [4]:
```
df.describe()
```

Out[4]:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

The median for Age is at 28 years old.

In [5]:
```
# Convert Sex text column to 0 and 1
df["Sex"] = df["Sex"].map({"male": 0, "female": 1})

# Find the median age and fill the missing (NaN) values
median_age = df["Age"].median()
df["Age"] = df["Age"].fillna(median_age)
```

Step 3: Select Features (X) and Target (y)

Separate the attributes you want the model to learn from (the features) from the outcome you want to predict (Survived). Make sure you do not include Survived or any columns from it in the X variables to prevent data errors (leakage).

In [6]:
```
# Choose 2-4 numeric features
features = ["Pclass", "Age", "Sex", "Fare"]
X = df[features]

# Target variable
y = df["Survived"]
```

Step 4: Create an Honest Test Set (70/30 Split)

Split your dataset into a training group (70%) to teach your classifier and a testing group (30%) to evaluate how well it performs on unseen passengers. The 2 parts for training purposes and remaining to evaluate our model. making test_size=0.3

In [7]:
```
# Split the data 70% train and 30% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Step 5: Fit the Classifier and Calculate Test Accuracy

(either a shallow DecisionTreeClassifier or a KNeighborsClassifier), train it using your training dataset, and print out score.

In [8]:
```
# Initialize a shallow Decision Tree ( max_depth=3 to keep it shallow)
# From lecture 7.1 What Machine Learning Actually Is. "Features, labels an a three line predict, cre
model = DecisionTreeClassifier(max_depth=3, random_state=42)

# Train the model
model.fit(X_train, y_train)
```

```
# Make predictions on the test set
y_pred = model.predict(X_test)

# Calculate accuracy score
accuracy = accuracy_score(y_test, y_pred)
print(f"Your Classifier TEST Accuracy: {accuracy * 100:.2f}%")
```

```
Your Classifier TEST Accuracy: 80.60%
```

The 80.6% Test Accuracy is good, however we will identify common model errors by analyzing the True Survival dataset versus the Predicted Survival dataset

Step 6: Identify Common Model Errors

To write a sentence about where the model drops accuracy, where the true label does not equal the prediction.

In [9]:
```
# Combine features, true answers, and predictions to look at the errors
analysis_df = X_test.copy()
analysis_df["True_Survival"] = y_test
analysis_df["Predicted_Survival"] = y_pred

# Filter out rows where the model was wrong
errors = analysis_df[analysis_df["True_Survival"] != analysis_df["Predicted_Survival"]]

# Display details or groupings of errors to spot trends
print(errors.groupby(["Sex", "Pclass"]).size())
```

```
Sex  Pclass
0    1         20
     2          2
     3         12
1    2          5
     3         13
dtype: int64
```

From the data we know that male is 0 and female is 1. This data is showing the exact number of mistakes this model made on the test set, grouped by gender and ticket class. The model is indicating that it commonly makes an error, mostly with men in 1st class (counted 20) and women in 3rd class (counted 13). **The model made 34 total mistakes on men, because it predicted they would die based on their gender, but they actually survived due to their premium ticket class.** Also the data says **the model made 18 total mistakes on women, because it predicted they would survive based on their gender, but they tragically died due to their third class ticket location on the ship.**

Behzad Ghabaei - End Of Lab

In [ ]:
```
```